Kubernetes Admission Security Operators

Policy Enforcement via Admission Controllers & Kyverno

k8s-kyverno-demo

Kubernetes Security

March 2026

Outline

1. Introduction
2. Background
3. Methods
4. Results
5. Discussion

Introduction — The Shared-Cluster Problem

Why cluster-wide policy enforcement?

- Many teams share a single Kubernetes cluster
- `kubectl apply` gives users broad power
- Common misconfigurations in practice:
  - `nginx:latest` — unpinned, drifts silently
  - No CPU/memory limits → noisy-neighbour DoS
  - Privileged containers → full node escape
  - Missing labels → invisible to monitoring
- **Goal:** automatically *reject* non-compliant workloads before they reach `etcd`

Container Escape Risk

A container running as `root` with `privileged: true` and a `hostPath` volume can:

- Read/write host filesystem
- Access node credentials
- Pivot to the control plane

This is a **critical** threat in multi-tenant clusters.

Background — Kubernetes Admission Controller Pipeline

ValidatingAdmissionWebhook

- Called *before* persisting any resource
- Returns `allowed: true/false` + reason
- Kyverno registers itself as one automatically

Why Kyverno?

- Policies are plain YAML — no Rego/Go required
- Runs fully in-cluster as a Deployment
- Covers: validate, mutate, generate, verify-image

Methods — Environment Setup

Toolchain

- Minikube (local single-node cluster)
- Helm (Kyverno package manager)
- kubectl (resource management)
- Makefile for reproducibility

One-command setup:

```
$ make start
# Starts Minikube, installs Kyverno
# via Helm, applies all policies
```

```
# Makefile excerpt
start:
  minikube start
  helm repo add kyverno \
    https://kyverno.github.io/kyverno/
  helm repo update
  helm upgrade --install kyverno \
    kyverno/kyverno \
    -n kyverno --create-namespace
  # Wait up to 20 retries x 5 s
  kubectl wait --for=condition=available\
    deployment --all -n kyverno \
    --timeout=120s
  kubectl apply -f policies/
```

Methods — Policy 1: Disallow `latest` Image Tag

policies/policy.yaml

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-latest-tag
spec:
  validationFailureAction: Enforce
  rules:
    - name: check-image-tag
      match:
        resources:
          kinds:
            - Pod
      validate:
        message: "latest tag is not allowed"
        pattern:
          spec:
            containers:
              - image: "!*:latest"
```

Rationale

- `latest` is mutable — image content can change between pulls silently
- Supply-chain attack vector: a compromised registry can replace `latest` with malicious content
- Breaks reproducibility: two identical manifests may run different binaries

Pattern logic: `"!*:latest"` rejects any image whose tag is exactly `latest`; explicit semver tags (`nginx:1.25.3`) are accepted.

Methods — Policies 2 & 3: Labels and Resource Limits

policies/policy-requires-labels.yaml

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-app-label
spec:
  validationFailureAction: Enforce
  rules:
    - name: check-label
      match:
        resources:
          kinds: [Pod]
      validate:
        message: "app label is required"
        pattern:
          metadata:
            labels:
              app: "?*"
```

policies/policy-resources.yaml

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-resources
spec:
  validationFailureAction: Enforce
  rules:
    - name: check-resources
      match:
        resources:
          kinds: [Pod]
      validate:
        message: >
          CPU and memory limits required
        pattern:
          spec:
            containers:
              - resources:
                  limits:
                    memory: "?*"
                    cpu: "?*"
```

Methods — Policy 4: Prevent Container Escapes

policies/policy-security-context.yaml

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-privileged-containers
spec:
  validationFailureAction: Enforce
  rules:
    - name: no-privileged
      match:
        resources:
          kinds: [Pod]
      validate:
        message: >
          Privileged containers not allowed.
          Use a non-root, non-privileged
          security context.
        pattern:
          spec:
            containers:
              - securityContext:
                  privileged: "false | !exists
                    "
                  allowPrivilegeEscalation:
                    "false | !exists"
                  runAsNonRoot: true
```

Threat model

- `privileged: true` gives container *full* host capabilities (CAP_SYS_ADMIN)
- `hostPath` mounts expose the node FS including kubelets, SSH keys, tokens
- `runAsRoot` + writable FS → escape via `nsenter`, `chroot`, `/proc`
- `allowPrivilegeEscalation` enables `setuid` binaries inside container

Defence-in-depth

This policy complements Pod Security Admission (PSA) and Linux seccomp/AppArmor profiles, stopping misconfigured workloads *at entry*.

Methods — Policy 5: Block Sensitive Host Mounts

policies/policy-sensitive-mounts.yaml

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-sensitive-mounts
spec:
  validationFailureAction: Enforce
  rules:
    - name: check-sensitive-paths
      match:
        resources:
          kinds: [Pod]
      validate:
        message: >
          Sensitive host path mount detected.
          Mounting docker socket, host creds
          or /proc is forbidden.
        deny:
          conditions:
            any:
            - key: "{{ request.object.spec\
  .volumes[].hostPath.path }}"
              operator: AnyIn
              value:
              - /var/run/docker.sock
              - /var/run/crio/crio.sock
```

Attack scenario

- Mount `/var/run/docker.sock` ⇒ full Docker daemon access from inside container; attacker runs `docker run --privileged` to get root on the node
- Mount `/etc/kubernetes` ⇒ reads admin `kubeconfig` / service-account tokens
- Mount `/proc` ⇒ read memory of any host process via `/proc/<pid>/mem`

Rejection output

```
$ kubectl apply -f spy-pod.yaml

Error from server: admission webhook
"validate.kyverno.svc-fail" denied
the request:

resource Pod/default/spy-pod was
blocked due to the following policies:
```

Methods — Policy 6: Prevent Host PID / Shared Procfs

policies/policy-host-namespaces.yaml

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-host-namespaces
spec:
  validationFailureAction: Enforce
  rules:
    - name: check-host-pid-ipc
      match:
        resources:
          kinds: [Pod]
      validate:
        message: >
          hostPID/hostIPC/hostNetwork share
          the host's process table, IPC, and
          network -- all forbidden in this
          cluster.
        pattern:
          spec:
            =(hostPID): "false"
            =(hostIPC): "false"
            =(hostNetwork): "false"
```

Attack scenario

- `hostPID: true` ⇒ container sees *all* host PIDs; attacker reads `/proc/<pid>/environ` for secrets/tokens and can `ptrace` or `kill` arbitrary processes
- `hostNetwork: true` ⇒ sniff raw node traffic; reach the metadata API (`169.254.169.254`) for cloud credentials
- `hostIPC: true` ⇒ attach to shared memory segments of other workloads

Rejection output

```
$ kubectl apply -f proc-spy-pod.yaml

Error from server: admission webhook
"validate.kyverno.svc-fail" denied
the request:

resource Pod/default/proc-spy was
blocked due to the following policies:
```

Results — Non-Compliant Workload & Rejection

test/deployment.yaml (non-compliant)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-test
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest   # VIOLATION
        # No resource limits   # VIOLATION
        # No securityContext   # VIOLATION
```

Admission controller output:

```
$ kubectl apply -f test/deployment.yaml

Error from server:
admission webhook
"validate.kyverno.svc-fail" denied
the request:

resource Deployment/default/nginx-test
was blocked due to the following
policies:

disallow-latest-tag:
  check-image-tag: latest tag is
  not allowed

require-resources:
  check-resources: CPU and memory
  limits are required
```

The request never reaches `etcd` — zero runtime exposure.

Results — Policy Coverage Matrix

Policy	Threat Addressed	Action	Scope
`disallow-latest-tag`	Supply-chain / image drift	Enforce	ClusterPolicy
`require-app-label`	Observability gap	Enforce	ClusterPolicy
`require-resources`	Resource starvation / DoS	Enforce	ClusterPolicy
`disallow-privileged`	Container escape (root/caps)	Enforce	ClusterPolicy
`disallow-sensitive-mounts`	Docker socket / host cred theft	Enforce	ClusterPolicy
`disallow-host-namespaces`	Procfs spy / network sniff	Enforce	ClusterPolicy

Compliant workload (accepted):

- Image: `nginx:1.25.3`
- Labels: `app: nginx`
- Limits: `cpu: 200m, memory: 128Mi`
- `runAsNonRoot: true, privileged: false`

Blocked scenarios (rejected):

- Any `:latest` tagged image
- Pod without app label
- Container without CPU/memory limits
- Privileged or root container
- `hostPath` volume mounts

Discussion — Findings, Limitations & Future Work

Key Findings

- Kyverno integrates in minutes via Helm with *zero* custom code
- `validationFailureAction: Enforce` blocks at the API layer — no workload ever starts
- Policies are auditable plain YAML, suitable for GitOps (ArgoCD/Flux)
- Container escape prevention closes critical multi-tenant attack paths

Limitations

- Policies currently target `Pod` kind only; `Deployment`/`DaemonSet` require additional rules
- No image vulnerability scanning (CVE-level)
- Kyverno is a SPOF unless deployed in HA

Future Work

- **Image verification** — cosign signatures + Kyverno `verifyImages` rules
- **Vulnerability scanning** — Trivy Operator + Kyverno `generate` to block known-CVE images
- **Mutation rules** — auto-inject resource defaults and non-root security context
- **Network isolation** — `NetworkPolicy` generation per namespace via Kyverno
- **Runtime enforcement** — Falco / Tetragon for post-admission behavioural anomaly detection
- **Policy-as-Code CI** — `kyverno test` in GitHub Actions / GitLab CI pipelines

Conclusion

Summary

Kubernetes admission controllers — implemented here via Kyverno — provide a **preventive, policy-as-code layer** that stops non-compliant and potentially malicious workloads *before* they are scheduled, with no runtime overhead.

Implemented

- Image tag pinning
- Mandatory labels
- Resource limits
- Privilege/escape prevention

Mechanisms

- ValidatingAdmissionWebhook
- Kyverno ClusterPolicy
- Helm + Makefile automation
- One-command reproducibility

Takeaway

- Shift security **left**: enforce at admission, not at runtime
- Policies are code — version, review, test them
- Layer defences: PSA + Kyverno + runtime tools
- Start in `Audit` mode, graduate to `Enforce`

Repository: `k8s-kyverno-demo` **Stack:** Minikube · Kyverno · Helm · kubectl